
Minimum Spanning Trees



Review: What's a tree?

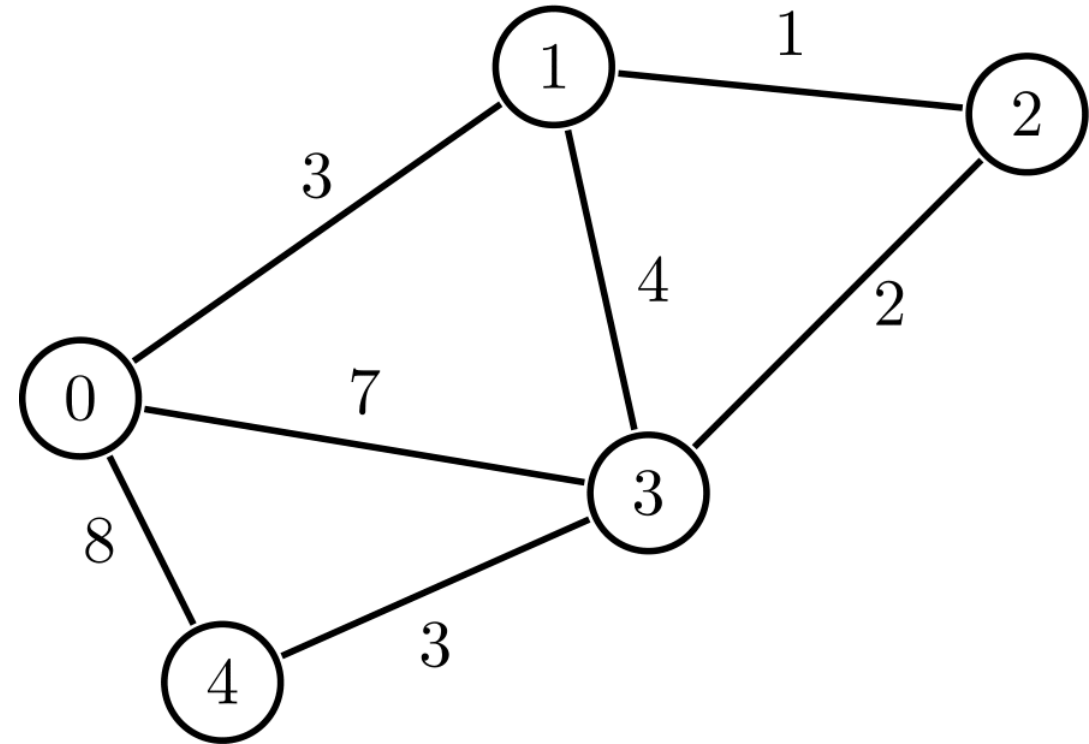
- An acyclic graph.

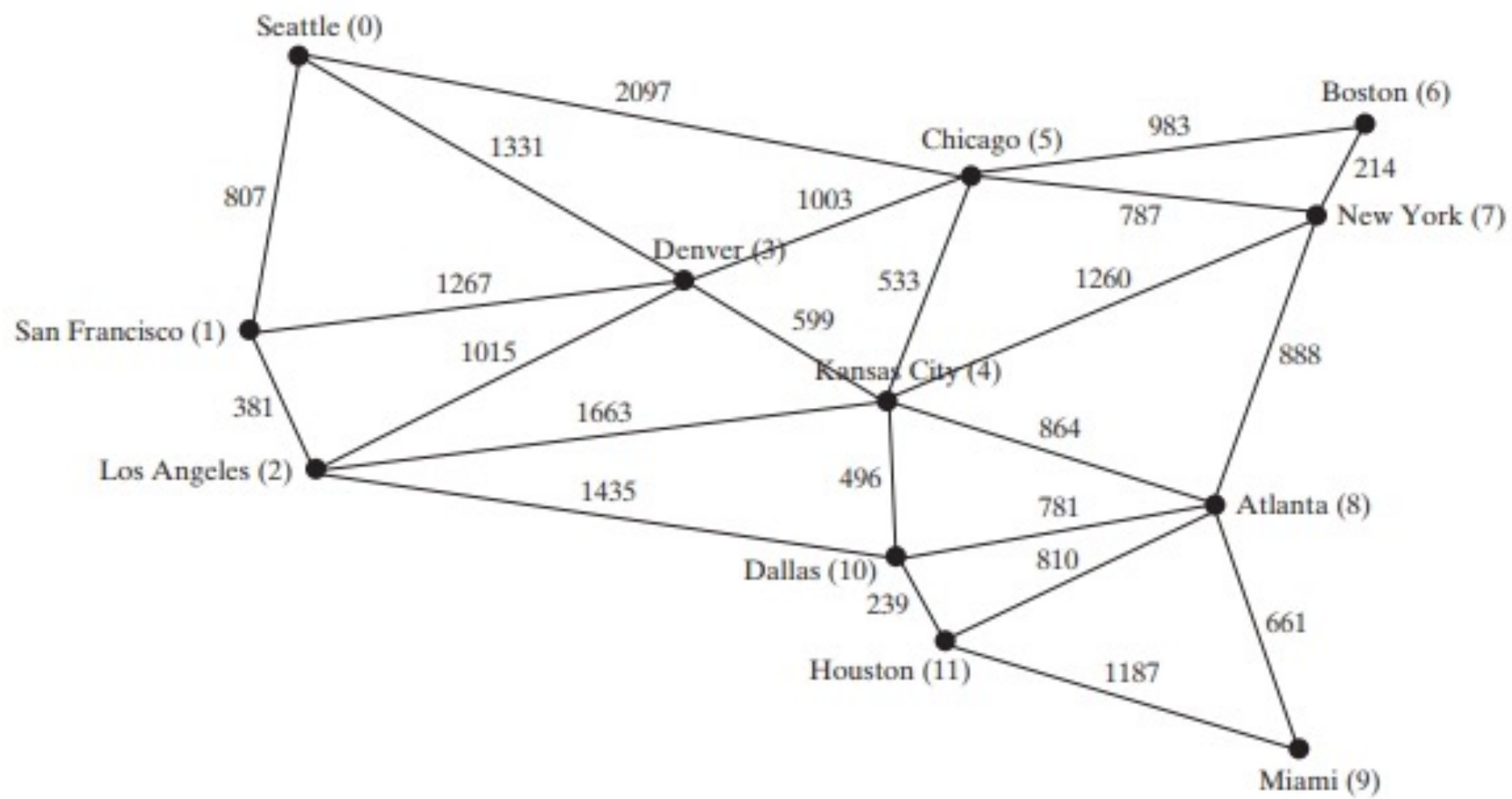
Review: What's a spanning tree?

- A tree which includes every vertex in the graph.

Review: What's a weighted graph?

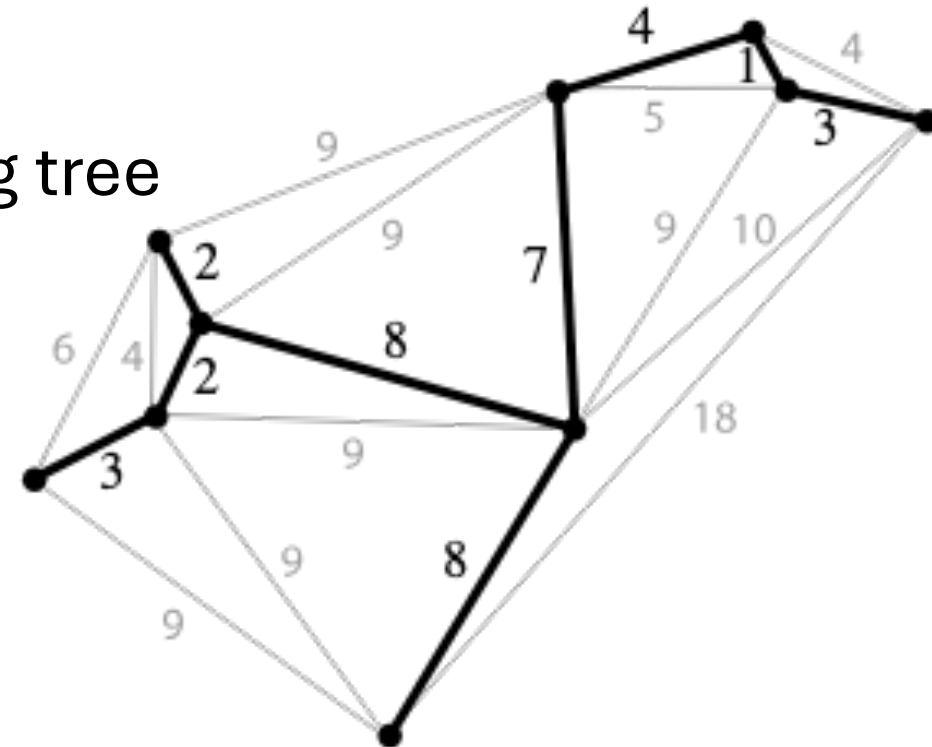
- Graph with numbers on the edges





What's a minimum spanning tree?

- A spanning tree with minimum weight
- We could not find a smaller-weight spanning tree



How to get MST?

- Two main algorithms (though there are more)
 - Prim's
 - Kruskal's



Robert Clay Prim



Joseph Kruskal

Main Ideas – Prim's

- Choose any vertex.
- Add next vertex, choose smallest weighted edge among options available, as long as choosing next vertex won't introduce a cycle

Minimum edge weight data structure	Time complexity (total)
adjacency matrix, searching	$O(V ^2)$
binary heap and adjacency list	$O((V + E) \log V) = O(E \log V)$
Fibonacci heap and adjacency list	$O(E + V \log V)$


```

function Prim(vertices, edges) is
  for each vertex in vertices do
    cheapestCost[vertex]  $\leftarrow \infty$ 
    cheapestEdge[vertex]  $\leftarrow$  null

  explored  $\leftarrow$  empty set
  unexplored  $\leftarrow$  set containing all vertices

  startVertex  $\leftarrow$  any element of vertices
  cheapestCost[startVertex]  $\leftarrow 0$ 

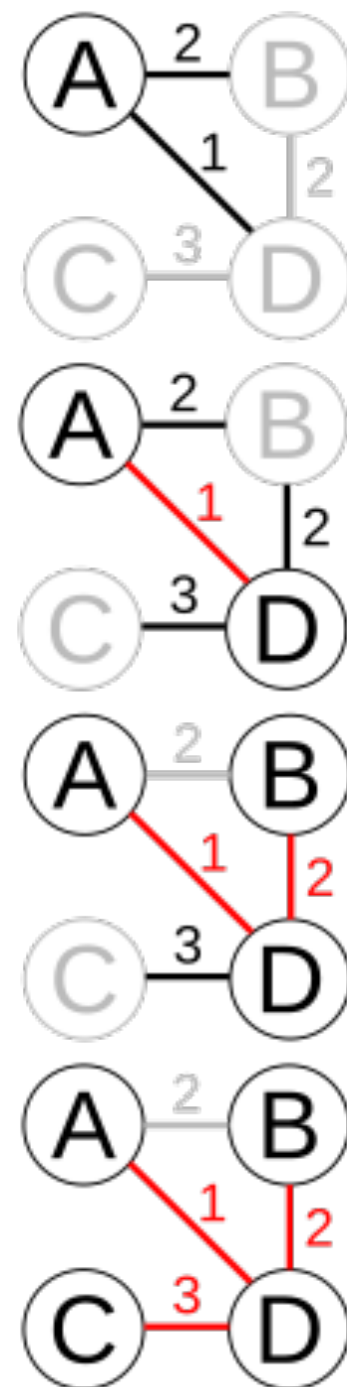
  while unexplored is not empty do
    // Select vertex in unexplored with minimum cost
    currentVertex  $\leftarrow$  vertex in unexplored with minimum cheapestCost[vertex]
    unexplored.remove(currentVertex)
    explored.add(currentVertex)

    for each edge (currentVertex, neighbor) in edges do
      if neighbor in unexplored and weight(currentVertex, neighbor) <
cheapestCost[neighbor] then
        cheapestCost[neighbor]  $\leftarrow$  weight(currentVertex, neighbor)
        cheapestEdge[neighbor]  $\leftarrow$  (currentVertex, neighbor)

  resultEdges  $\leftarrow$  empty list
  for each vertex in vertices do
    if cheapestEdge[vertex]  $\neq$  null then
      resultEdges.append(cheapestEdge[vertex])

  return resultEdges

```



Do an example

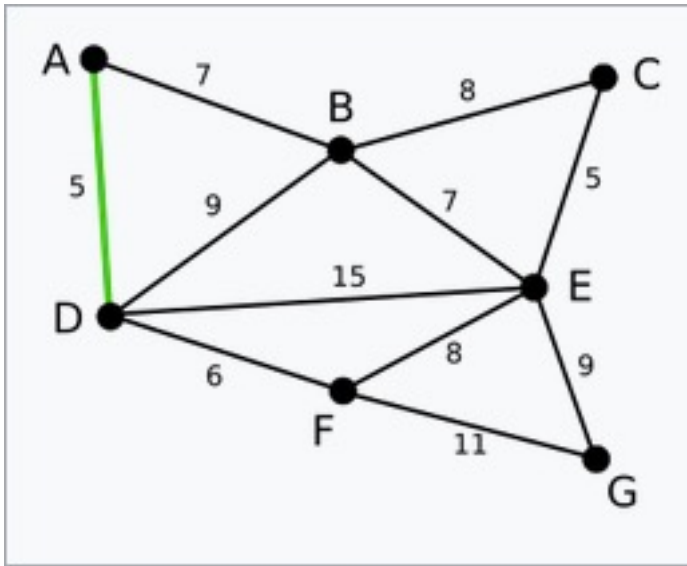
You try

- With your neighbors:
- Choose 2-3 graphs from the sheet you've been given
- Draw on the board
- Run Prim's algorithm on the graphs

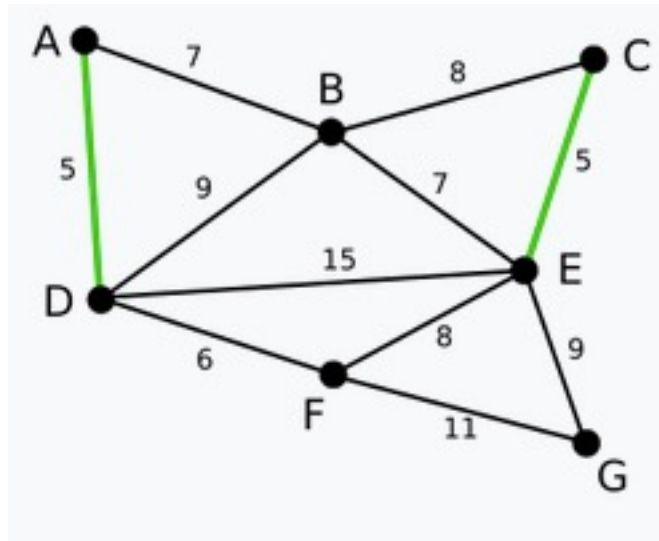
Main Ideas - Kruskal's

- Sort edges by weight
- Instead of choosing vertices, choose smallest weighted *edge* in the entire graph. [Choose cheapest edge and ask: are these two vertices already in the subgraph?]
 - Include if not, as long as the edge won't introduce a cycle
- Build entire tree using Union-Find data structure
- Complexity: $O(|E| \log |E|)$

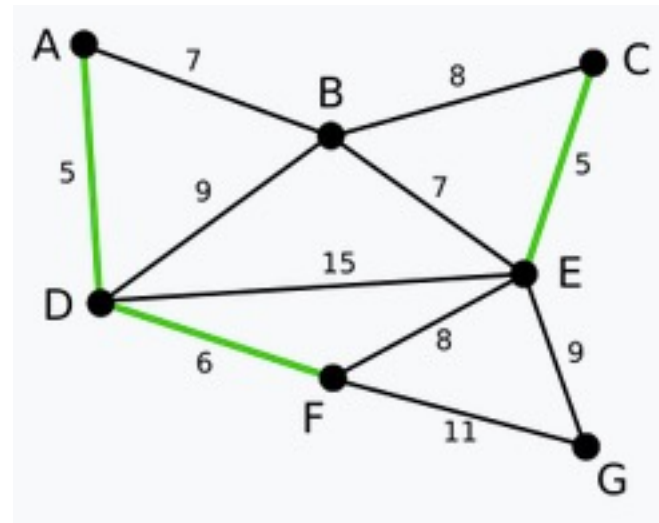
```
function Kruskal(Graph G) is  
    F :=  $\emptyset$   
    for each v in G.Vertices do  
        MAKE-SET(v)  
  
    for each {u, v} in G.Edges ordered by increasing weight({u, v}) do  
        if FIND-SET(u)  $\neq$  FIND-SET(v) then  
            F := F  $\cup$  { {u, v} }  
            UNION(FIND-SET(u), FIND-SET(v))  
    return F
```



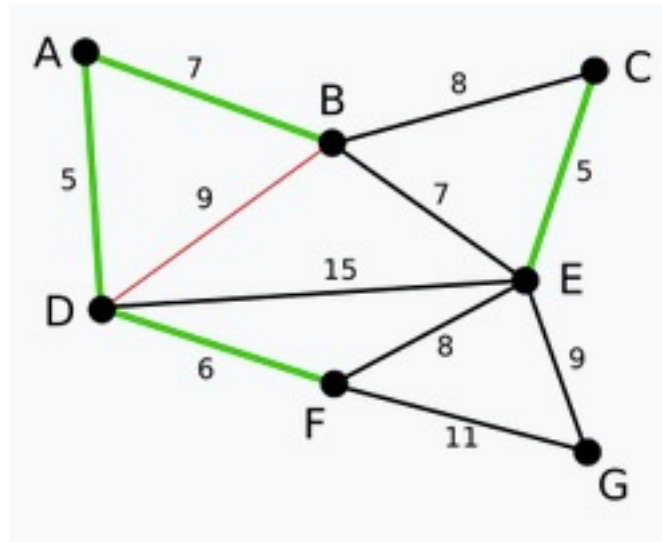
a



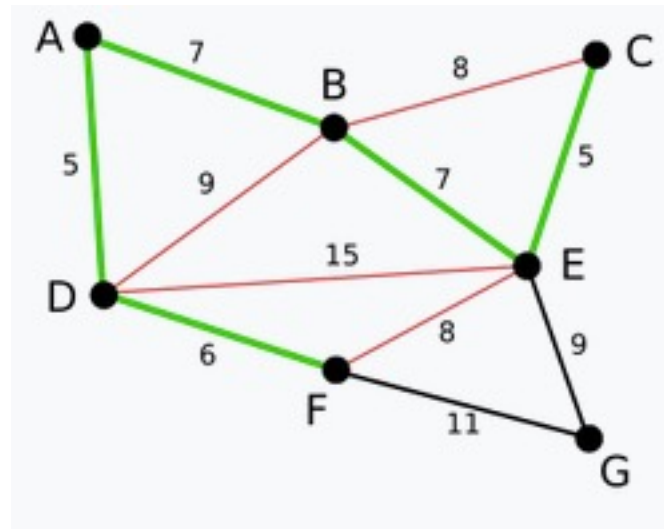
b



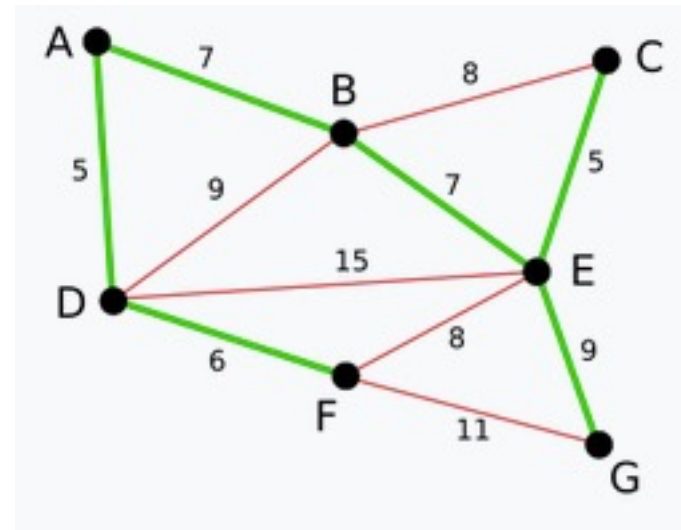
c



d



e



f

Do an example

You try

- With your neighbors:
- Choose 2-3 graphs from the sheet you've been given
- Draw on the board
- Run Kruskal's algorithm on the graphs

Cool-down

- I am constructing a unified water pipeline and need to connect every major city to my system. The weights of my graph represent distance. How can I set up my pipeline using the fewest miles of material possible?

